



操作系统实验

题 目	:	实验8
专业名称	:	计算机科学与技术
授课教师	:	陈鹏飞
学生姓名	:	陈安康
学 号	:	22320021
实验地点	:	实验中心D402
日 期	:	2024/6/11

一、系统调用的实现流程

首先保存完现场后，通过int 0x80来调用0x80中断，里面存放有中断处理函数的代码选择子和偏移地址，这时硬件会进行特权级检查，确保调用中断的用户程序的特权级可以调用该中断。因为中断处理函数的代码段选择子被加载入cs寄存器，所以CPL随之发生了变化，因此实现的特权级的转移。在硬件作用下，CPU将TSS中对应的高优先级的栈的段选择子和栈指针加载入ss和esp寄存器中，此时的栈不再试原来的低特权级栈，而是变成了高特权级栈。因此，系统调用过程中特权级的转变实际上是从中断描述符这里完成的。

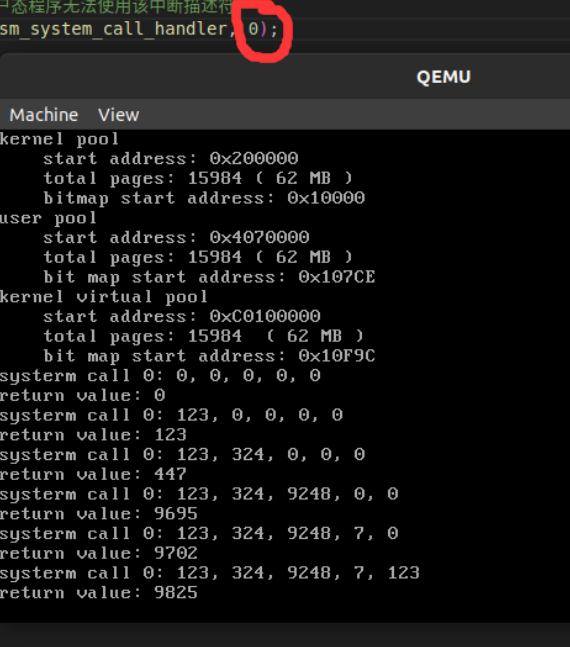
然后在中断处理函数中，依据系统调用号查找系统调用表里面的系统调用函数，打开终端，执行函数，执行完毕后关中断，恢复现场，执行中断返回，低特权级栈的信息在进入中断前被保存在高特权级栈中，因此执行 `iret` 后，低特权级栈的SS和ESP变可以被恢复。回到低特权级的用户程序中。这样我们就完成了系统调用。

由于中间进行特权级转移时栈也进行了转移，因此我们不能把系统调用的参数放在栈上，这样在栈进行转移时就找不到参数了，因此我们是通过寄存器传递参数的方法，先将放在低特权级栈上的参数放在寄存器中，然后执行中断，切换到高特权级的栈上后，再将寄存器中的参数压栈，调用系统调用号指定的系统调用函数。

在没有实现进程前，教程中的“系统调用的实现”部分的展示没有那么好。一方面是此时教程中一切代码都是在内核中运行的，代码段选择子的CPL一直都是0，一直处于内核态，因此对于中断描述符的DPL的设置用途仅仅就此部分来看并没有体现出来。因此，在教程给出的代码1中，无论中断描述符的DPL设置成多少，都可以运行：

```
// 代码段选择子默认是DPL=0的平坦模式代码段选择子，DPL=3，否则用户态程序无法使用该中断描述符
interruptManager.setInterruptDescriptor(0x80, (uint32)asm_system_call_handler, 0);
}

bool SystemService::setSystemCall(int index, int function)
{
    system_call_table[index] = function;
    return true;
}
```



```
Machine View
kernel pool
  start address: 0x200000
  total pages: 15984 ( 62 MB )
  bitmap start address: 0x10000
user pool
  start address: 0x4070000
  total pages: 15984 ( 62 MB )
  bit map start address: 0x107CE
kernel virtual pool
  start address: 0xC0100000
  total pages: 15984 ( 62 MB )
  bit map start address: 0x10F9C
system call 0: 0, 0, 0, 0, 0
return value: 0
system call 0: 123, 0, 0, 0, 0
return value: 123
system call 0: 123, 324, 0, 0, 0
return value: 447
system call 0: 123, 324, 9248, 0, 0
return value: 9695
system call 0: 123, 324, 9248, 7, 0
return value: 9702
system call 0: 123, 324, 9248, 7, 123
return value: 9825
```

还有就是TSS进行特权级转移的部分，由于这部分大部分操作由硬件完成，而我的GDB没有办法查看TSS，所以对此很是疑惑，虽然教程讲的很细致，但由于没有实现用户进程的原因，目前全部运行在内核中，所有选择子都是用的一套，我通过GDB追踪的段选择子在系统调用过程中完全没有任何变化（这是自然的，以代码段选择子为例，所有的代码全部运行在内核中，用的代码段选择子肯定是一样的，尽管中途在中断描述符处进行了所谓的“切换”，切换后代码段选择子也是原来的代码段选择子，没有改变，其余同理），这给我这种看一部分教程完成一部分实验的人来说造成了一段时间的困惑。

因此建议助教可以在“系统调用的实现”这一部分进行说明，即由于目前没有实现用户进程，导致教程中的一些解释没有在代码运行过程中体现出来，不要过度纠结于此。

二、进程的实现：

以下代码位于src/3中。

在进程实现后，特权级的作用能够表现出来了，当再次将中断描述符的DPL改为0后，运行程序则会直接闪退（这没办法截图，所以不进行展示了），说明特权级检查不允许低特权级的用户进程访问使用高特权级的中断，特权级起作用了。

之前实验值预留的共享部分的作用在这个实验中有了体现，这次我们从bootloader时就打开的分页机制，所以所有的地址都对应发生了一些变化，转移到了3GB - 4GB，以防寻址错误。

之前我们实现的线程一直是在内核态运行的，而现在创建的进程是在用户态下运行的，通过对放置在GDT的段选择子特权级的设置来实现创建的进程运行在用户态下。教程中用户进程的段选择子都是所有用户进程共用的，因此将其定义在ProgramManager中进行初始化，送入GDT。

为了能够成功完成通过系统调用实现从用户态到内核态的转换，我们还要实现TSS。通过TSS提供高特权级栈的段选择子和栈指针来完成栈的转移。TSS是存储在内存中的。因为对于它的大部分操作是通过硬件完成的，为了能让CPU找到它，我们需要把它的地址等相关信息放在一个特定的寄存器中，即TR寄存器。值得注意的是，ss0所有的用户进程是共用的，但每个进程的esp0是不同的，这意味着每个进程都会维护一个高特权级栈，通过进程调度的代码不难看到，这个栈就是在PCB中，而且还是在PCB的底部开始。

```
28 void ProgramManager::activateProgramPage(PCB *program)
29 {
30     int paddr = PAGE_DIRECTORY;
31
32     if (program->pageDirectoryAddress)
33     {
34         tss.esp0 = (int)program + PAGE_SIZE;
35         paddr = memorymanager.vaultzpadaddr(program->pageDirectoryAddress);
36     }
37
38     asm_update_cr3(paddr);
39 }
```

除此之外，进程与线程的最大不同还有进程有独立的虚拟空间。这意味着它需要有自己独立的页目录表与页表，和自己的虚拟地址池，这些信息全部放在PCB中进行存储。这也要求在线程切换时，需要进行页目录表的切换，这不同于上一个实验从头到尾全部用的一个页目录表，进行切换也很简单，只需要重新赋值cr3即可（这里要求依据虚拟地址找到页目录表的物理地址，上一个实验已经实现了这个方法，这里不赘述）。

接下来就是加载进程的工作了。因为我们是在内核代码中，一直处在内核态里面，要想加载用户态进程，那么我们就一定会经过从内核态到用户态的这一步，这一步的实现只能通过中断返回。因此我们要用中断返回的方法到达用户态。注意到我们之前的线程会在栈上留有一个线程函数的位置，这里我们将他利用起来，实现了一个load_process函数，这个函数对真正需要执行的进程函数进行一个封装，load_process函数的作用就是实现高特权级向低特权级的转移，通过iret返回到我们需要执行的进程函数上。为了实现这一点，维护了一个栈，将用户进程需要的环境都放在栈上，然后通过pop来实现环境转换，iret实现特权级的转移和目标函数的加载。

在这个load_process这个部分我最初有个疑问，那就是在asm_start_process函数中，这里使用了popad来把InterruptStack栈中的寄存器恢复，由于设置的esp是0，那么如果直接popad，难道不会让esp的值变为0从而导致换栈错误吗？

对于此进行GDB调试追踪：

InterruptStack栈中的内容，前面9个是通用寄存器和gs段寄存器的值设置为0，后面的依次往下，0xc0020aa4是第一个进程函数的首地址：

```
终端
../src/kernel/program.cpp
317         {
318             printf("can not build process!\n");
319             process->status = ProgramStatus::DEAD;
320             asm_halt();
321         }
322         interruptStack->esp += PAGE_SIZE;
323         interruptStack->ss = programManager.USER_STACK_SELECTOR;
324
B->325         asm_start_process((int)interruptStack);
326     }
327
328     void ProgramManager::activateProgramPage(PCB *program)
329     {
330         int paddr = PAGE_DIRECTORY;

remote Thread 1.1 In: load_process L325 PC: 0xc0020989
--Type <RET> for more, q to quit, c to continue without paging--c
0x08049000
(gdb) x/2xw 0xc0020aa4
0xc0020aa4 <first_process(>: 0xfb1e0ff3 0x83e58955
(gdb) x/17xw 0xc00267c
0xc00267c: Cannot access memory at address 0xc00267c
(gdb) x/17xw 0xc00267dc
0xc00267dc <PCB_SET+12220>: 0x00000000 0x00000000 0x00000000
0x00000000
0xc00267ec <PCB_SET+12236>: 0x00000000 0x00000000 0x00000000
0x00000000
0xc00267fc <PCB_SET+12252>: 0x00000000 0x00000033 0x00000033
0x00000033
0xc002680c <PCB_SET+12268>: 0xc0020aa4 0x0000002b 0x00000202
0x08049000
0xc002681c <PCB_SET+12284>: 0x0000003b
(gdb) 
```

跳进asm_start_process函数中执行，会发现当执行完popad之后，esp寄存器的值没有如预想般变为0，而是依旧在这个栈上（对比上下两个图会发现esp依旧在InterruptStack栈上）：

```
终端
+
./src/utils/asm_utils.asm
39      asm_start_process:
40          ; jmp $
B+ 41      mov eax, dword[esp+4]
42      mov esp, eax
43      popad
>44      pop gs;
45      pop fs;
46      pop es;
47      pop ds;
48
49      iret
50
51      ; void asm_ltr(int tr)
52      asm_ltr:

remote Thread 1.1 In: asm_start_process L44 PC: 0xc00227a7
(gdb) i r eax
eax          0x0          0
(gdb) i r esp
esp          0xc00267fc    0xc00267fc <PCB_SET+12252>
(gdb) i r ebp
ebp          0x0          0x0
(gdb) x/4xw 0xc00267fc
0xc00267fc <PCB_SET+12252>:  0x00000000  0x00000033  0x00000033  0x00000033
0x00000033
(gdb) x/13xw 0xc00267fc
0xc00267fc <PCB_SET+12252>:  0x00000000  0x00000033  0x00000033  0x00000033
0x00000033
0xc002680c <PCB_SET+12268>:  0xc0020aa4  0x0000002b  0x00000202
0x08049000
0xc002681c <PCB_SET+12284>:  0x0000003b  0xc00277c0  0xfb1e0ff3
0x83e58955
0xc002682c <PCB_SET+12300>:  0xec8308ec
(gdb) █
```

后来经过查找网上资料发现这是popad命令的设计造成的，在执行popad过程中，遇到本应赋给esp的值时，它会跳过不对esp进行赋值。也就是说，**popad不会影响esp!** 但是它会更改ebp的值（这在上图也进行了展示）。由此也可以说明为什么变量命名时会将本应赋给esp的值命名为esp_dummy。

除此之外，还需要明确一点，这个load_process函数以及为此维护的interruptStack都是“一次性”的，只在初次加载时运行这个函数，以后就不用了。它们会被高特权栈覆盖掉。

以下通过GDB来说明TSS在特权级转换中的作用：

TSS主要是提供高特权级的栈段选择子以及栈指针的。**低特权级栈是从虚拟空间重新分配的一页，高特权级栈是在PCB上的。**下图展示了当调用int 80 中断时esp发生了变化，从低特权级栈的栈顶0x8048fd0 跳转到了TSS提供的高特权级栈的栈顶0xc0025888。

```

../src/utils/asm_utils.asm
79      lgdt [ASM_GDTR]
80
81      pop esi
82      pop ebx
83      pop ebp
84
85      ret
86      ; int asm_system_call_handler();
87      asm_system_call_handler:
88          push ds
>89      push es
90          push fs
91          push gs
92          pushad
93
94          push eax
95
96          ; ^&^ &^ $^ $^ ss^$^ (^ %^ %^ (^
97
98          mov eax, DATA_SELECTOR
99          mov ds, eax
100         mov es, eax
101
remote Thread 1.1 In: asm system call handler
Breakpoint 4 at 0xc0022863: file ../src/utils/asm_utils.asm, line 130.
(gdb) c
Continuing.

Breakpoint 4, asm_system_call () at ../src/utils/asm_utils.asm:130
(gdb) i r esp
esp      0x8048fd0      0x8048fd0
(gdb) b 146
Breakpoint 5 at 0xc002287d: file ../src/utils/asm_utils.asm, line 146.
(gdb) c
Continuing.

Breakpoint 5, asm_system_call () at ../src/utils/asm_utils.asm:146
(gdb) ni
asm_system_call_handler () at ../src/utils/asm_utils.asm:89
(gdb) i r esp
esp      0xc0025888      0xc0025888 <PCB_SET+8168>
(gdb)

```

下图展示了TSS.esp0和栈中的内容，可以发现当前栈的栈就是esp0所指向的栈，栈中目前存了ds寄存器的值，返回地址以及低特权级的栈指针等其他信息：

```

86      ; int asm_system_call_handler();
87      asm_system_call_handler:
88          push ds
>89      push es
90          push fs
91          push gs
92          pushad
93
94          push eax
95
96          ; ^&^ &^ $^ $^ ss^$^ (^ %^ %^ (^
97
98          mov eax, DATA_SELECTOR

remote Thread 1.1 In: asm system call handler
(gdb) p /x tss.esp0
$3 = 0xc00258a0
(gdb) x/16xw 0xc0025888
0xc0025888 <PCB_SET+8168>: 0x00000033 0xc002287f 0x0000002b 0x00000212
0xc0025898 <PCB_SET+8184>: 0x08048f08 0x0000003b 0x00000000 0x00000000
0xc00258a8 <PCB_SET+8200>: 0x00000000 0x00000000 0x00000000 0x00000000
0xc00258b8 <PCB_SET+8216>: 0x00000000 0x00000000 0x00000000 0x00000000
(gdb) x/i 0xc002287f
0xc002287f <asm_system_call+28>: pop edi

```

下图展示了在调用系统调用函数时，栈没有改变(严谨点说是栈的总体位置没有改变)，依旧是在原来的高特权级栈上面：


```
23
24     int syscall_0(int first, int second, int third, int forth, int flfth)
B+>25     {
26         printf("system call 0: %d, %d, %d, %d, %d\n",
27             first, second, third, forth, flfth);
28         return first + second + third + forth + flfth;
29     }
30
31     void syscall_1(int first, int second, int third, int forth, int flfth) {
32         printf("syscall_1 is called!");
33     }
34
35     void first_process()
36     {
37         asm_system_call(0, 132, 324, 12, 124);

```

remote Thread 1.1 In: syscall_0

```
esp      0xc0025888      0xc0025888 <PCB_SET+8168>
(gdb) c
Continuing.

Breakpoint 6, asm_system_call_handler () at ../src/utils/asm_utils.asm:115
(gdb) i r esp
esp      0xc0025848      0xc0025848 <PCB_SET+8104>
(gdb) c
Continuing.

Breakpoint 7, syscall_0 (first=132, second=324, third=12, forth=124, flfth=0) at ../src/kernel/setup.cpp:25
(gdb) i r esp
esp      0xc0025844      0xc0025844 <PCB_SET+8100>
(gdb) x/1xw 0xc0025844
0xc0025844 <PCB_SET+8100>: 0xc002284d
(gdb) x/l 0xc002284d
0xc002284d <asm_system_call_handler+38>: cll
(gdb)

```

下图展示了在完成系统调用后，栈指针返回到低特权级栈：

```
138
139     mov eax, [ebp + 2 * 4]
140     mov ebx, [ebp + 3 * 4]
141     mov ecx, [ebp + 4 * 4]
142     mov edx, [ebp + 5 * 4]
143     mov esi, [ebp + 6 * 4]
144     mov edi, [ebp + 7 * 4]
145
B+ 146     int 0x80
147
>148     pop edi
149     pop esi
150     pop edx
151     pop ecx
152     pop ebx
153     pop ebp
154
155     ret
156
157     ; void asm_init_page_reg(int *directory);
158     asm_init_page_reg:
159     push ebp
Visual Studio Code    mov ebp, esp

```

remote Thread 1.1 In: asm system call

```
(gdb) ni
(gdb) ni
asm_system_call_handler () at ../src/utils/asm_utils.asm:116
(gdb) b 128
Breakpoint 9 at 0xc0022862: file ../src/utils/asm_utils.asm, line 128.
(gdb) i r esp
esp      0xc0025848      0xc0025848 <PCB_SET+8104>
(gdb) c
Continuing.

Breakpoint 9, asm_system_call_handler () at ../src/utils/asm_utils.asm:128
(gdb) i r esp
esp      0xc002588c      0xc002588c <PCB_SET+8172>
(gdb) ni
asm_system_call () at ../src/utils/asm_utils.asm:148
(gdb) i r esp
esp      0x8048fb8      0x8048fb8
(gdb)

```

值得注意的是，虽然我们的调度代码本质是没有变化的，但是操作的esp却发生了很大的不同：在asm_switch_thread中，正常情况下，保存在PCBstack指针中的esp变成了低特权栈的栈顶，以便恢复。

因为系统调用时已经打开了中断，因此意味着系统调用过程中允许触发进程切换，如果在执行系统调用过程中发生了进程切换，那么此时保存在PCBstack指针中的esp变成了高特权级栈的栈顶，以便恢复。

借由以上也可以推断出来，**tss上的esp0保存的是高特权级栈的初始的栈顶，只在初始时将此值赋值给esp，如果系统调用过程中发生了进程切换，那么tss并不负责保护上下文的作用，而是交由PCB的stack指针来完成。**因此保证了即使在系统调用过程中发生了进程切换，也不会由于信息丢失而导致错误。

以下是验证例子，说明在系统调用过程中发生进程切换也能保证运行正确。

稍微修改0号系统调用的代码，添加一个延迟器，并且在延迟器前后分别打印信息，然后创建1号系统调用，实现教程中“添加更多系统调用”的任务，打印一则信息，将这两个系统调用在系统调用表中初始化，在主线程中创建两个进程调用这两个系统调用。由于延迟器的作用，0号系统调用在执行时会触发进程切换，换到第二个进程调用1号系统调用打印信息，换回第一个进程后，由于PCB中的stack保存了高特权级栈的esp，所以可以恢复上下文继续执行：

```
int syscall_0(int first, int second, int third, int forth, int fifth)
{
    printf("system call 0: %d, %d, %d, %d, %d\n",
           first, second, third, forth, fifth);
    int c = 0xffffffff;
    while(c--) {
    }
    printf("system call 0 res: %d\n",
           first + second + third + forth + fifth);
    return first + second + third + forth + fifth;
}

void syscall_1(int first, int second, int third, int forth, int fifth) {
    printf("syscall_1 is called!\n");
}

void first_process()
{
    asm_system_call(0, 132, 324, 12, 124);
    asm_halt();
}

void second_process() {
    asm_system_call(1,0,0,0,0);
    asm_halt();
}

void first_thread(void *arg)
{
    printf("start process\n");
    programManager.executeProcess((const char *)first_process, 1);
    // programManager.executeProcess((const char *)first_process, 1);
    // programManager.executeProcess((const char *)first_process, 1);
    programManager.executeProcess((const char *)second_process, 1);
    asm_halt();
}

// GDB console output:
(gdb) run
Starting program: /home/.../...
start process\n
system call 0: 132, 324, 12, 124, 0
syscall_1 is called!
system call 0 res: 592
```

GDB追踪结果如下：


```
终端
../src/kernel/setup.cpp
26     printf("system call 0: %d, %d, %d, %d, %d\n",
27           first, second, third, forth, fifth);
28     int c = 0xffffffff;
29     while(c--) {
30
31     }
B+>32     printf("system call 0 res: %d\n",
33           first + second + third + forth + fifth);
34     return first + second + third + forth + fifth;
35 }
36
37 void syscall_1(int first, int second, int third, int forth, in
38 printf("syscall_1 is called!\n");

remote Thread 1.1 In: syscall_0 L32 PC: 0xc0020ab8
(gdb) c
Continuing.

Breakpoint 2, syscall_0 (first=132, second=324, third=12, forth=124, fifth=0)
at ../src/kernel/setup.cpp:32
(gdb) p /x programManager -> running -> stack
$1 = 0xc00257f8
(gdb) i r esp
esp          0xc00258a8          0xc00258a8 <PCB_SET+8072>
(gdb) p /x tss.esp0
$2 = 0xc0025920
(gdb) █

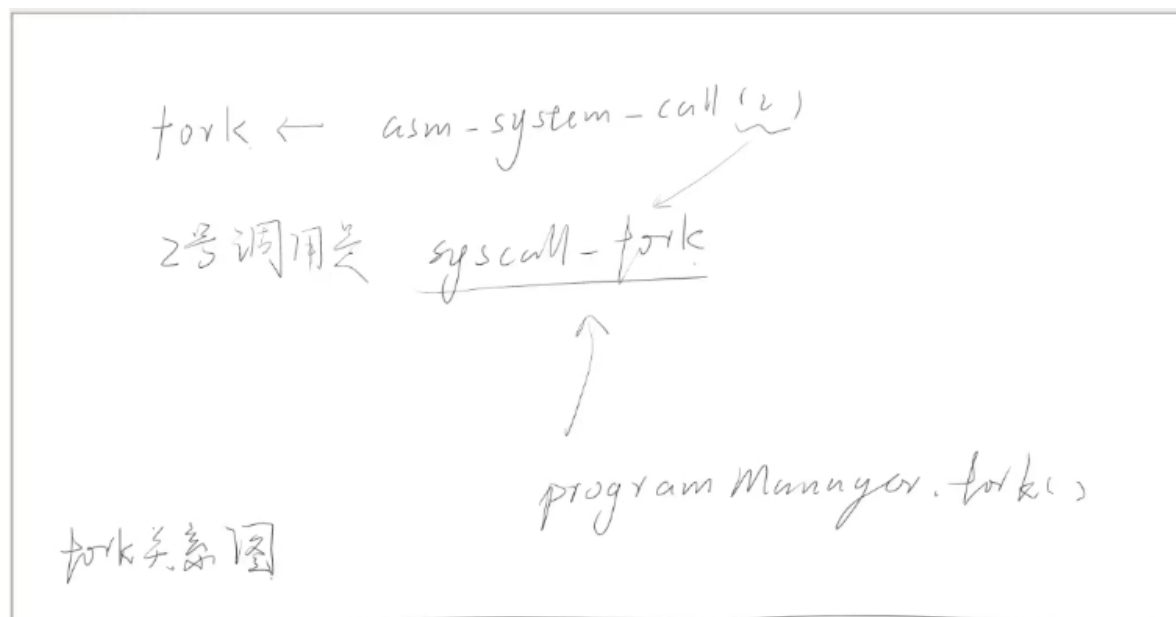
r *)first_process_1):
```

stack与esp之间的差异是因为恢复环境而pop了寄存器导致的，通过以上例子说明是stack保存了esp的值，使得高级特权栈也可以进行上下文恢复。并且上文也展示了1号系统调用执行的正确性。

三、fork函数：

首先分析一下fork函数的实现逻辑。

fork函数的关系如下：



箭头的含义是“封装与”。可以看见fork函数套了好几层皮，最终是运行的programManager的fork函数。

在programManager的fork函数里面，child的PCB的创建和判断（**可以通过判断PageDirectoryAddress来区别线程与进程是因为PCB初始时全部被0填充，因为线程没有设置本应存放PageDirectoryAddress的内存，所以线程的PageDirectoryAddress天然为0**），最重要的就是copyProcess函数，现在我们主要分析这个函数。

我们分成两个部分分析这个函数，我称为子进程启动结构构造部分和内存资源复制部分。

1.子进程启动结构构造部分：

这部分主要解决子进程返回父进程的返回点的问题，以及PCB上资源的复制，包括0特权级栈的复制以及虚拟池等资源的复制。

需要完成fork要求的父子进程同时执行同一段代码的效果，需要做到两点：父子进程初始环境是一样的，父子进程执行的代码相同。

由于代码在内核中，父子进程之间天然代码共享。由于子进程和父进程需要从同一个返回点开始往下执行，要完成这一个目标构造了如上的结构首先，需要明确返回的点在哪儿，其实，**我们需要返回到父进程执行的asm_system_call_handler应该返回的位置，即asm_system_call调用int 80之后的语句**。此时如果父进程环境已经全部复制到子进程的话，那么子进程运行时就像是接着父进程的同一返回点继续执行（这里会通过中断处理函数等函数一步步往下返回，由于栈等环境已经完全复制，那么会返回到父进程运行的那个进程函数，并且环境一样）。

环境0特权级栈，PCB、虚拟地址池、页目录表、页表及其指向的物理页。页目录表、页表及其指向的物理页的复制在第二部分完成，这一部分我们要复制0特权级栈，PCB、虚拟地址池（也就是PCB上的环境）到子进程。

那么怎么做到找到父进程的返回点，及恢复父进程在asm_system_call_handler中的环境呢？由此我们在子进程的PCB上构造了如下结构：

首先来分析一下各个结构的通用功能。首先是InterruptStack，这里的InterruptStack功能与之前的InterruptStack一样，**即与asm_start_process配套使用可以返回到InterruptStack保存的eip执行并且可以恢复环境**，但作用有很大的不同。在之前已经通过GDB展示过，父进程在运行asm_system_call_handler以及syscall_fork时用的都是0特权级栈。子进程不关心syscall_fork的栈存储内容，因为这就是用来构造它的，它关心的是调用syscall_fork以前asm_system_call_handler存储在栈上的环境，只要能把栈的这部分复制到子进程中就能找到返回点了。

很“巧”的是，在父进程调用syscall_fork之前形成的栈结构与之前为了实现高特权级到低特权级转换加载PCB而创建的一次性InterruptStack结构如出一辙。那么就意味着我们可以复用这一部分的代码。

首先要将这部分栈内容从父进程复制到子进程，然后需要搭配asm_start_process，因为我们是通过正常的进程切换来换到子进程的，而进程切换时用到的asm_switch_thread对栈结构是有要求的所以我们还要在这一部分的基础上构造一个符合asm_switch_thread的栈：

```

// 复制进程0级栈
ProcessStartStack *childpss =
    (ProcessStartStack *)((int)child + PAGE_SIZE - sizeof(ProcessStartStack));
ProcessStartStack *parentpss =
    (ProcessStartStack *)((int)parent + PAGE_SIZE - sizeof(ProcessStartStack));
memcpy(parentpss, childpss, sizeof(ProcessStartStack));
// 设置子进程的返回值为0
childpss->eax = 0;

// 准备执行asm_switch_thread的栈的内容
child->stack = (int *)childpss - 7;
child->stack[0] = 0;
child->stack[1] = 0;
child->stack[2] = 0;
child->stack[3] = 0;
child->stack[4] = (int)asm_start_process;
child->stack[5] = 0; // asm_start_process 返回地址
child->stack[6] = (int)childpss; // asm_start_process 参数

```

这里顺手设置了子进程的返回值为0，这个值不是走调用syscall_fork得到的，子进程也没有调用过syscall_fork，而是我们人为设置的。因为函数返回值默认保存在eax中，这个值就会被当做子进程“调用”syscall_fork后的返回值进行处理。

这里教程还隐含着给stack指针赋值了要求的esp的值，在上面的手绘结构图里面我写了这个能成功还有关键的就是子进程的PCB里面保存的esp的值要正好指向构造的栈的栈顶，这个值不能复制父进程的，而是需要自己设置，教程中通过先划分空间再赋值的方式保证了stack指针保存的是栈顶。

然后我们再复制PCB中的信息和虚拟池的信息即可：

```

// 设置子进程的PCB
child->status = ProgramStatus::READY;
child->parentPid = parent->pid;
child->priority = parent->priority;
child->ticks = parent->ticks;
child->ticksPassedBy = parent->ticksPassedBy;
strcpy(parent->name, child->name);

// 复制用户虚拟地址池
int bitmapLength = parent->userVirtual.resources.length;
int bitmapBytes = ceil(bitmapLength, 8);
memcpy(parent->userVirtual.resources.bitmap, child->userVirtual.resources.bitmap, bitmapBytes);

```

2.内存资源复制部分：

这一部分我们不仅要求资源复制，还要求转到子进程后，对应资源的虚拟地址是完全一样的。那么直接在当下的父进程虚拟地址中进行操作，不能保证虚拟地址是一致的。所以我们创建个共享的中转页，先将父进程的内容复制给它，然后跳转到子进程空间上，将中转页的内容复制给子进程由此就可以实现内存资源的复制。

```

char *buffer = (char *)memoryManager.allocatePages(AddressPoolType::KERNEL, 1);
if (!buffer)
{
    child->status = ProgramStatus::DEAD;
    return false;
}

// 子进程页目录表物理地址
int childPageDirPaddr = memoryManager.vaddr2paddr(child->pageDirectoryAddress);
// 父进程页目录表物理地址
int parentPageDirPaddr = memoryManager.vaddr2paddr(parent->pageDirectoryAddress);
// 子进程页目录表指针(虚拟地址)
int *childPageDir = (int *)child->pageDirectoryAddress;
// 父进程页目录表指针(虚拟地址)
int *parentPageDir = (int *)parent->pageDirectoryAddress;

// 子进程页目录表初始化
memset((void *)child->pageDirectoryAddress, 0, 768 * 4);

```

```

// 复制页目录表
for (int i = 0; i < 768; ++i)
{
    // 无对应页表
    if (!(parentPageDir[i] & 0x1))
    {
        continue;
    }

    // 从用户物理地址池中分配一页，作为子进程的页目录项指向的页表
    int paddr = memoryManager.allocatePhysicalPages(AddressPoolType::USER, 1);
    if (!paddr)
    {
        child->status = ProgramStatus::DEAD;
        return false;
    }
    // 页目录项
    int pde = parentPageDir[i];
    // 构造页表的起始虚拟地址
    int *pageTableVaddr = (int *) (0xffc00000 + (i << 12));

    asm_update_cr3(childPageDirPaddr); // 进入子进程虚拟地址空间

    childPageDir[i] = (pde & 0x00000fff) | paddr;
    memset(pageTableVaddr, 0, PAGE_SIZE);

    asm_update_cr3(parentPageDirPaddr); // 回到父进程虚拟地址空间
}

```

```

// 复制页表和物理页
for (int i = 0; i < 768; ++i)
{
    // 无对应页表
    if (!(parentPageDir[i] & 0x1))
    {
        continue;
    }
    // 计算页表的虚拟地址
    int *pageTableVaddr = (int *) (0xffc00000 + (i << 12));
    // 复制物理页
    for (int j = 0; j < 1024; ++j)
    {
        // 无对应物理页
        if (!(pageTableVaddr[j] & 0x1))
        {
            continue;
        }
        // 从用户物理地址池中分配一页，作为子进程的页表项指向的物理页
        int paddr = memoryManager.allocatePhysicalPages(AddressPoolType::USER, 1);
        if (!paddr)
        {
            child->status = ProgramStatus::DEAD;
            return false;
        }
        // 构造物理页的起始虚拟地址
        void *pageVaddr = (void *) ((i << 22) + (j << 12));
        // 页表项
        int pte = pageTableVaddr[j];
        // 复制出父进程物理页的内容到中转页
        memcpy(pageVaddr, buffer, PAGE_SIZE);
        asm_update_cr3(childPageDirPaddr); // 进入子进程虚拟地址空间
        pageTableVaddr[j] = (pte & 0x00000fff) | paddr;
        // 从中转页中复制到子进程的物理页
        memcpy(buffer, pageVaddr, PAGE_SIZE);
        asm_update_cr3(parentPageDirPaddr); // 回到父进程虚拟地址空间
    }
}

```

这部分的内容的原理基本上上一个实验已经全部分析过了，所以不再赘述。需要注意的是我们的子进程的虚拟地址是先在父进程中构造好之后（子进程的虚拟地址都是构造出来的），再转到子进程虚拟地址空间下，然后**先添加对应项（这时候将申请的物理页与构造的子进程下的虚拟地址联系起来），再复制**，不然的话寻址会找不到产生错误。

为什么有些地方不用虚拟池分配呢？因为没法这样做，因为子进程的虚拟池已经完全复制了父进程的虚拟池，用虚拟池分配的话由于那些位置的资源位已经显示被占用了（实际上没有），所以分到的虚拟地址肯定不会和父进程的——对应。

那只要不先复制虚拟池不就好了？这样也用不了虚拟池，因为虚拟池的分配代码如下：

```
int MemoryManager::allocateVirtualPages(enum AddressPoolType type, const int count)
{
    int start = -1;

    if (type == AddressPoolType::KERNEL)
    {
        start = kernelVirtual.allocate(count);
    }
    else if (type == AddressPoolType::USER)
    {
        start = programManager.running->userVirtual.allocate(count);
    }

    return (start == -1) ? 0 : start;
}
```

这个用户进程的虚拟池分配只认当前正在运行的用户进程的虚拟池，也就是父进程的虚拟池，根本没法操作子进程的虚拟池，所以无论如何，都用不了虚拟池分配，只能通过构造虚拟地址来完成。

至此，fork的代码逻辑结构分析完毕。

下面是GDB追踪调试：

这里教程将write() 系统调用封装在了printf中，所以可以直接调用printf进行io访问。

下图展示父进程在执行到asm_system_call_handler时的栈情况。这个栈与InteruptStack结构一致：

```
终端
../src/utls/asm_utils.asm
86      ; int asm_system_call_handler();
87      asm_system_call_handler:
B+ 88      push ds
89      push es
90      push fs
91      push gs
92      pushad
93
>94      push eax
95
96      ; ^&^ &^ $^ $^ ss^$^ (^ %^ %^ (^
LibreOffice Writer
98      mov eax, DATA_SELECTOR
99      mov ds, eax
100     mov es, eax
101
102     mov eax, VIDEO_SELECTOR
103     mov gs, eax
104
105     pop eax
106
107     ; ^%^ &^ %^ &^
108     push edi

remote Thread 1.1 In: asm system call handler
asm_system_call_handler () at ../src/utls/asm_utils.asm:91
(gdb) nt
asm_system_call_handler () at ../src/utls/asm_utils.asm:92
(gdb) nt
asm_system_call_handler () at ../src/utls/asm_utils.asm:94
(gdb) i r esp
esp      0xc0025f3c      0xc0025f3c <PCB_SET+8124>
(gdb) x/17xw 0xc0025f3c
0xc0025f3c <PCB_SET+8124>:  0x00000000      0x00000000      0x08048fac      0xc0025f5c
0xc0025f4c <PCB_SET+8140>:  0x00000000      0x00000000      0x00000000      0x00000002
0xc0025f5c <PCB_SET+8156>:  0x00000000      0x00000033      0x00000033      0x00000033
0xc0025f6c <PCB_SET+8172>:  0xc0022e6f      0x0000002b      0x00000216      0x08048f98
0xc0025f7c <PCB_SET+8188>:  0x0000003b
(gdb)
```

下图展示是初始切换到子进程时栈中的内容。对比可以发现栈顶的17项内容与上图基本一致（除了保存 **eax** 的位置），并且也保存了 `asm_start_process` 函数的地址。在 `InterruptStack` 中保存了 `asm_system_call` 的地址，这是子进程应当返回执行的位置。与预期一致：


```

../src/utls/asm_utils.asm
190      ret
191
192      ; void asm_switch_thread(PCB *cur, PCB *next);
193      asm_switch_thread:
B+ 194      push ebp
195      push ebx
196      push edi
197      push esi
198
199      mov eax, [esp + 5 * 4]
200      mov [eax], esp ; ^$^ ^ %^ %^ &^ &^ )^ %^ CB^$^ / ^ $^ $^ ^ ^ &^ $
201
202      mov eax, [esp + 6 * 4]
203      mov esp, [eax] ; ^&^ &^ &^ %^ ' ^ $^ ur^&^ %^ &^ %^ ext^&^
204
>205      pop esi
206      pop edi
207      pop ebx
208      pop ebp
209
210      sti
211      ret
212      ; int asm_interrupt_status();

remote Thread 1.1 In: asm_switch_thread
(gdb) ni
(gdb) ni
(gdb) ni
asm_switch_thread () at ../src/utls/asm_utils.asm:205
(gdb) i r esp
esp          0xc0026f20          0xc0026f20 <PCB_SET+12192>
(gdb) x/24xw 0xc0026f20
0xc0026f20 <PCB_SET+12192>:  0x00000000  0x00000000  0x00000000  0x00000000  0x00000000
0xc0026f30 <PCB_SET+12208>:  0xc0022dc0  0x00000000  0xc0026f3c  0x00000000
0xc0026f40 <PCB_SET+12224>:  0x00000000  0x08048fac  0xc0025f5c  0x00000000
0xc0026f50 <PCB_SET+12240>:  0x00000000  0x00000000  0x00000000  0x00000000
0xc0026f60 <PCB_SET+12256>:  0x00000033  0x00000033  0x00000033  0xc0022e6f
0xc0026f70 <PCB_SET+12272>:  0x0000002b  0x00000216  0x08048f98  0x0000003b
(gdb) x/i 0xc0022dc0
0xc0022dc0 <asm_start_process>:  mov     eax,DWORD PTR [esp+0x4]
(gdb) x/i 0xc0022e6f
0xc0022e6f <asm_system_call+28>:  pop     edi
(gdb)

```

这个展示了子进程从asm_switch_thread执行返回操作进入了asm_start_process。

```

./src/utls/asm_utils.asm
31     ASM_TEMP dd 0
32     ; void asm_update_cr3(int address)
33     asm_update_cr3:
34         push eax
35         mov eax, dword[esp+8]
36         mov cr3, eax
37         pop eax
38         ret
39     asm_start_process:
40         ; jmp $
>41     mov eax, dword[esp+4]
42     mov esp, eax
43     popad
44     pop gs;
45     pop fs;
46     pop es;
47     pop ds;
48
49     iret
50
51     ; void asm_ltr(int tr)
52     asm_ltr:
53     ltr word[esp + 1 * 4]

remote Thread 1.1 In: asm_start_process
(gdb) ni
(gdb) ni
(gdb) ni
(gdb) ni
asm_switch_thread () at ./src/utls/asm_utils.asm:205
(gdb) ni
asm_switch_thread () at ./src/utls/asm_utils.asm:206
(gdb) ni
asm_switch_thread () at ./src/utls/asm_utils.asm:207
(gdb) ni
asm_switch_thread () at ./src/utls/asm_utils.asm:208
(gdb) ni
asm_switch_thread () at ./src/utls/asm_utils.asm:210
(gdb) ni
asm_switch_thread () at ./src/utls/asm_utils.asm:211
(gdb) ni
asm_start_process () at ./src/utls/asm_utils.asm:41
(gdb) █
```

由asm_start_process经过IntterruptStack返回到了asm_system_call调用完int 80之后:

```

../src/utils/asm_utils.asm
138
139      mov eax, [ebp + 2 * 4]
140      mov ebx, [ebp + 3 * 4]
141      mov ecx, [ebp + 4 * 4]
142      mov edx, [ebp + 5 * 4]
143      mov esi, [ebp + 6 * 4]
144      mov edi, [ebp + 7 * 4]
145
146      int 0x80
147
>148      pop edi
149      pop esi
150      pop edx
151      pop ecx
152      pop ebx
153      pop ebp
154
155      ret
156
157      ; void asm_init_page_reg(int *directory);
158      asm_init_page_reg:
159      push ebp
160      mov ebp, esp

```

```

remote Thread 1.1 In: asm system call
asm_switch_thread () at ../src/utils/asm_utils.asm:211
(gdb) ni
asm_start_process () at ../src/utils/asm_utils.asm:41
(gdb) ni
(gdb) ni
(gdb) ni
asm_start_process () at ../src/utils/asm_utils.asm:44
(gdb) ni
asm_start_process () at ../src/utils/asm_utils.asm:45
(gdb) ni
asm_start_process () at ../src/utils/asm_utils.asm:46
(gdb) ni
asm_start_process () at ../src/utils/asm_utils.asm:47
(gdb) ni
asm_start_process () at ../src/utils/asm_utils.asm:49
(gdb) ni
asm_system_call () at ../src/utils/asm_utils.asm:148
(gdb) 

```

我们在之前copyProcess中人为设置了子进程的eax = 0，返回的pid = 0（第二张是另一次截的）：

```

../src/kernel/setup.cpp
23
24     int syscall_0(int first, int second, int third, int forth, int fifth)
25     {
26         printf("system call 0: %d, %d, %d, %d, %d\n",
27             first, second, third, forth, fifth);
28         return first + second + third + forth + fifth;
29     }
30
31     void first_process()
32     {
>33         int pid = fork();
34
35         if (pid == -1)
36         {
37             printf("can not fork\n");
38         }
39         else
40         {
41             if (pid)
42             {
43                 printf("I am father, fork reutrnr: %d\n", pid);
44             }
45             else

```

remote Thread 1.1 In: first process

asm_system_call () at ../src/utils/asm_utils.asm:148

(gdb) nl

(gdb) nl

(gdb) nl

(gdb) nl

(gdb) nl

(gdb) nl

(gdb) i r eax

eax 0x0 0

(gdb) nl

0xc0020fc0 in fork () at ../src/kernel/syscall.cpp:36

(gdb) nl

(gdb) nl

(gdb) nl

0xc0021032 in first_process () at ../src/kernel/setup.cpp:33

(gdb) p pid

\$1 = 0

(gdb) █

移入其中请按 Ctrl+C

```

25     {
26         printf("system call 0: %d, %d, %d, %d, %d\n",
27             first, second, third, forth, fifth);
28         return first + second + third + forth + fifth;
29     }
30
31     void first_process()
32     {
33         int pid = fork();
34
35         if (pid == -1)
36         {
37             printf("can not fork\n");
38         }
39         else
40         {
41             if (pid)
42             {
43                 printf("I am father, fork reutrnr: %d\n", pid);
44             }
45             else
46             {
47                 printf("I am child, fork return: %d, my pid: %d\n",

```

remote Thread 1.1 In: first process

Continuing.

Breakpoint 2, `asm_system_call ()` at `../src/utils/asm_utils.asm:130`

(gdb) c

Continuing.

Breakpoint 3, `asm_system_call ()` at `../src/utils/asm_utils.asm:148`

(gdb) c

Continuing.

Breakpoint 3, `asm_system_call ()` at `../src/utils/asm_utils.asm:148`

(gdb) c

Continuing.

Breakpoint 4, `first_process ()` at `../src/kernel/setup.cpp:35`

(gdb) p pid

\$3 = 0

(gdb)

下图为父进程返回后eax的值以及pid，作为对比，可以发现父子进程从asm_system_call开始返回的路径一致，它们的eax值不同，从而保证了返回的pid结果不同：

```
../src/kernel/setup.cpp
23
24     int syscall_0(int first, int second, int third, int forth, int fifth)
25     {
26         printf("system call 0: %d, %d, %d, %d, %d\n",
27             first, second, third, forth, fifth);
28         return first + second + third + forth + fifth;
29     }
30
31     void first_process()
32     {
>33         int pid = fork();
34
35         if (pid == -1)
36         {
37             printf("can not fork\n");
38         }
39         else
40         {
41             if (pid)
42             {
43                 printf("I am father, fork reutrnr: %d\n", pid);
44             }
45             else

```

remote Thread 1.1 In: first process
Breakpoint 3, asm_system_call () at ../src/utlis/asm_utils.asm:148
(gdb) i r eax
eax 0x2 2
(gdb) ni
(gdb) ni
(gdb) ni
(gdb) ni
(gdb) ni
(gdb) ni
(gdb) ni
0xc0020fc0 in fork () at ../src/kernel/syscall.cpp:36
(gdb) ni
(gdb) ni
(gdb) ni
0xc0021032 in first_process () at ../src/kernel/setup.cpp:33
(gdb) c

```
../src/kernel/setup.cpp
23
24     int syscall_0(int first, int second, int third, int forth, int fifth)
25     {
26         printf("system call 0: %d, %d, %d, %d, %d\n",
27             first, second, third, forth, fifth);
28         return first + second + third + forth + fifth;
29     }
30
31     void first_process()
32     {
33         int pid = fork();
34
>35         if (pid == -1)
36         {
37             printf("can not fork\n");
38         }
39         else
40         {
41             if (pid)
42             {
43                 printf("I am father, fork reutrnr: %d\n", pid);
44             }
45             else

```

remote Thread 1.1 In: first process
(gdb) ni
(gdb) ni
(gdb) ni
(gdb) ni
(gdb) ni
(gdb) ni
(gdb) ni
0xc0020fc0 in fork () at ../src/kernel/syscall.cpp:36
(gdb) ni
(gdb) ni
(gdb) ni
0xc0021032 in first_process () at ../src/kernel/setup.cpp:33
(gdb) p pid
\$1 = 0
(gdb) ni
(gdb) p pid
\$2 = 2
(gdb) █

至此，fork函数的分析全部完毕。

四、exit和wait函数：

1.exit函数：

exit()函数提供给进程主动结束运行的途径。其构成结构与fork函数类似，本质上是调用的programManager的exit()方法。exit的返回值放在PCB中，所以在PCB中专门设置一个变量用于存放返回值。整个exit()函数的作用为设置进程状态，若为进程则释放内存空间资源（通过对PageDirectoryAddress判断来区别线程和进程，线程不需要执行这个部分），调用调度函数。

进程退出后能够隐式地调用exit和此时的exit返回值是0，这是因为我们修改了load_process函数，在进程3特权级栈顶放入了如下结构：

0 (exit ()) 的默认参数)
0 (会被认作exit()的返回地址)
exit()的地址

这样，当进程执行完成后，此时exit()的地址会被当做返回地址，这样就进入exit()函数中，那么esp会指向下一项的0，这一项就会被当做exit () 的返回地址，而其下一项就会被认为是exit()的默认参数，这样，我们实现了进程的隐式调用exit函数返回并且exit的返回值是0。

对文件5中的代码运行GDB追踪如下，可以看到，first_process函数返回之前，站顶的前三个值是预期的样子：

```
>0xc0021294 <first_process()+94>      ret
0xc0021295 <second_thread(void*)>      endbr32
0xc0021299 <second_thread(void*)+4>    push    ebp
0xc002129a <second_thread(void*)+5>    mov     ebp,esp
0xc002129c <second_thread(void*)+7>    sub     esp,0x8
0xc002129f <second_thread(void*)+10>   sub     esp,0xc
0xc00212a2 <second_thread(void*)+13>   push    0xc0023299
0xc00212a7 <second_thread(void*)+18>   call    0xc0021941 <printf(char const*, ...)>
0xc00212ac <second_thread(void*)+23>   add     esp,0x10
0xc00212af <second_thread(void*)+26>   sub     esp,0xc
0xc00212b2 <second_thread(void*)+29>   push    0x0
0xc00212b4 <second_thread(void*)+31>   call    0xc00211af <exit(int)>
0xc00212b9 <second_thread(void*)+36>   add     esp,0x10
0xc00212bc <second_thread(void*)+39>   nop
0xc00212bd <second_thread(void*)+40>   leave
0xc00212be <second_thread(void*)+41>   ret
B+ 0xc00212bf <first_thread(void*)>    endbr32
0xc00212c3 <first_thread(void*)+4>    push    ebp
0xc00212c4 <first_thread(void*)+5>    mov     ebp,esp
0xc00212c6 <first_thread(void*)+7>    sub     esp,0x8
0xc00212c9 <first_thread(void*)+10>   sub     esp,0xc

remote Thread 1.1 In: first_process
(gdb) layout asm
(gdb) ni
(gdb) ni
(gdb) i r esp
esp                0x8048ff4      0x8048ff4
(gdb) x/3xw 0x8048ff4
0x8048ff4: 0xc00211af      0x00000000      0x00000000
(gdb) x/ti 0xc00211af
0xc00211af <exit(int)>:      endbr32
(gdb) |
```

再次执行，就跳转到了exit()函数中：

```

B+>0xc00211af <exit(int)>                                endbr32
0xc00211b3 <exit(int)+4>                                push    ebp
0xc00211b4 <exit(int)+5>                                mov     ebp,esp
0xc00211b6 <exit(int)+7>                                sub     esp,0x8
0xc00211b9 <exit(int)+10>                               sub     esp,0x8
0xc00211bc <exit(int)+13>                               push    0x0
0xc00211be <exit(int)+15>                               push    0x0
0xc00211c0 <exit(int)+17>                               push    0x0
0xc00211c2 <exit(int)+19>                               push    0x0
0xc00211c4 <exit(int)+21>                               push    DWORD PTR [ebp+0x8]
0xc00211c7 <exit(int)+24>                               push    0x3
0xc00211c9 <exit(int)+26>                               call    0xc00230c3 <asm_system_call>
0xc00211ce <exit(int)+31>                               add     esp,0x20
0xc00211d1 <exit(int)+34>                               nop
0xc00211d2 <exit(int)+35>                               leave
0xc00211d3 <exit(int)+36>                               ret
0xc00211d4 <syscall_exit(int)>                            endbr32
0xc00211d8 <syscall_exit(int)+4>                        push    ebp
0xc00211d9 <syscall_exit(int)+5>                        mov     ebp,esp
0xc00211db <syscall_exit(int)+7>                        sub     esp,0x8
0xc00211de <syscall_exit(int)+10>                       sub     esp,0x8
0xc00211e1 <syscall_exit(int)+13>                       push    DWORD PTR [ebp+0x8]
0xc00211e4 <syscall_exit(int)+16>                       push    0xc00346a0

remote Thread 1.1 In: exit
(gdb) layout asm
(gdb) ni
(gdb) ni
(gdb) i r esp
esp                0x8048ff4                0x8048ff4
(gdb) x/3xw 0x8048ff4
0x8048ff4:         0xc00211af                0x00000000                0x00000000
(gdb) x/i 0xc00211af
0xc00211af <exit(int)>:                endbr32
(gdb) ni

Breakpoint 3, exit (ret=0) at ../src/kernel/syscall.cpp:43
(gdb)

```

文件5中的代码执行结果如下：

```

QEMU

Machine  View

iPXE (http://ipxe.org) 00:03.0 CA00 PCI2.10 PnP PMM+07F8CB00+07ECCB00 CA00

Booting from Hard Disk...
total memory: 133038080 bytes ( 126 MB )
kernel pool
  start address: 0x200000
  total pages: 15984 ( 62 MB )
  bitmap start address: 0xC0010000
user pool
  start address: 0x4070000
  total pages: 15984 ( 62 MB )
  bit map start address: 0xC00107CE
kernel virtual pool
  start address: 0xC0100000
  total pages: 15984 ( 62 MB )
  bit map start address: 0xC0010F9C
start process
I am father
thread exit
I am child, exit

```

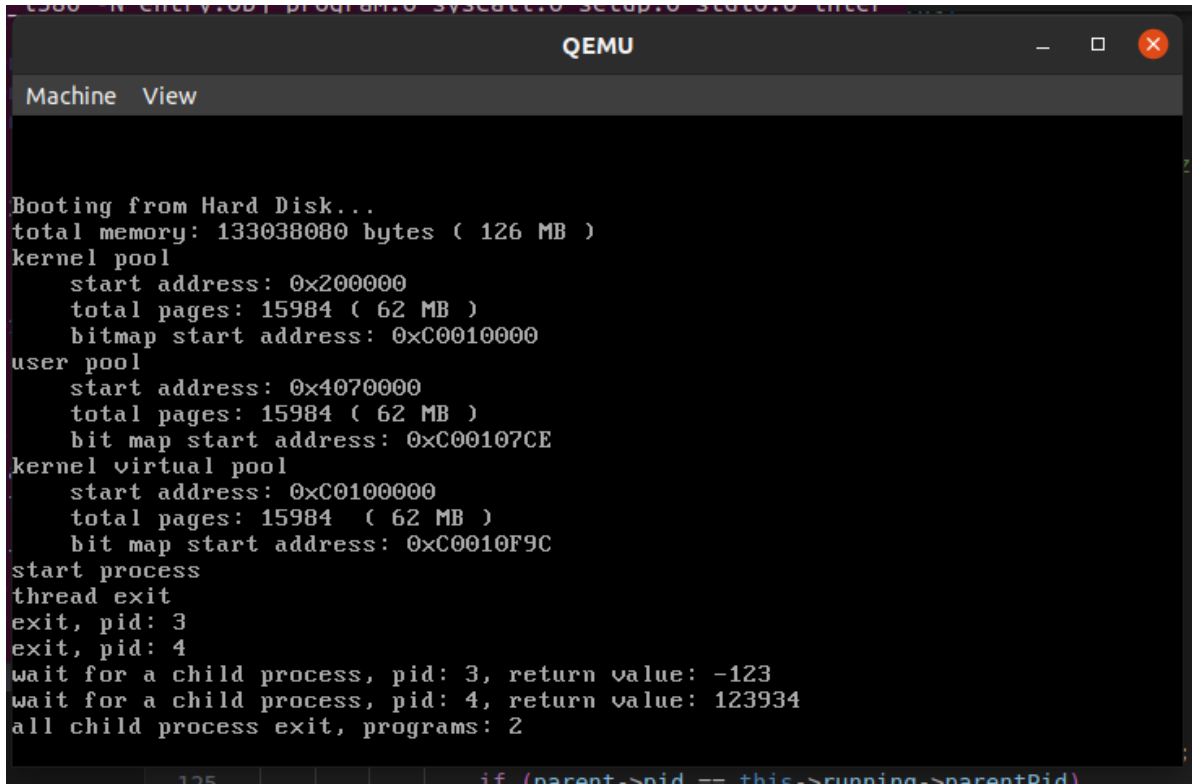
进程在 `exit` 的最后调用了 `schedule` 后，这个进程不会再被调度执行，这是因为进程的状态已经被设为 `DEAD`，调度函数只会将 `RUNNING` 状态的进程加入就绪队列，因此该进程不会再被调度了。

2.wait函数:

wait函数是用于父进程回收子进程的PCB，实现上，父进程在allPrograms列表里面查找自己的子进程如果找到的子进程状态为DEAD，则跳出循环，对这个进程，将子进程的返回值保存在retval里面，然后执行releasePCB，这个函数主要是将PCB资源位设置为空闲，然后allPrograms里面将这个进程删除掉。如果此时没有子进程，则返回-1，如果有子进程，但状态不为DEAD，那么就阻塞，执行调度函数调度其他函数（函数整体比较简单，也没什么特别的结构，就不截图了）。

需要注意的是，由于wait()函数的特性，我们一次最多只能回收一个子进程。

src/6执行结果如下：



```
QEMU
Machine View
Booting from Hard Disk...
total memory: 133038080 bytes ( 126 MB )
kernel pool
  start address: 0x200000
  total pages: 15984 ( 62 MB )
  bitmap start address: 0xC0010000
user pool
  start address: 0x4070000
  total pages: 15984 ( 62 MB )
  bit map start address: 0xC00107CE
kernel virtual pool
  start address: 0xC0100000
  total pages: 15984 ( 62 MB )
  bit map start address: 0xC0010F9C
start process
thread exit
exit, pid: 3
exit, pid: 4
wait for a child process, pid: 3, return value: -123
wait for a child process, pid: 4, return value: 123934
all child process exit, programs: 2
125 if (parent->pid == this->running->parentPid)
```

3.回收孤儿僵尸进程:

代码位于6文件夹中。

这里提供两个方法:

第一种方法:

第一种方法是修改调度函数，让调度函数承担这一功能。在进程状态为DEAD时，找到它的父进程进行判断，若其父进程状态也为DEAD，那么就回收PCB。如果没有父进程（两种可能，一、它本身就没有父进程，二、它的父进程被它父进程的父进程给回收了），也回收PCB。代码很简单，如下，位于schedule中:

```

else if (running->status == ProgramStatus::DEAD)
{
    // 回收线程，子进程留到父进程回收
    if(!running->pageDirectoryAddress) {
        releasePCB(running);
    }
    else {
        bool flag = true;
        ListItem *item_cake = this->allPrograms.head.next;
        while (item_cake)
        {
            PCB *parent = ListItem2PCB(item_cake, tagInAllList);
            if (parent->pid == this->running->parentPid)
            {
                if (parent->status == ProgramStatus::DEAD)
                {
                    releasePCB(running);
                    flag = false;
                    break;
                } else {
                    flag = false;
                    break;
                }
            }
            item_cake = item_cake->next;
        }
        if(flag)
            releasePCB(running);
    }
}

```

为了方便展示是否回收PCB，我在releasePCB函数中加入打印代码，打印回收的PCB的pid：

```

void ProgramManager::releasePCB(PCB *program)
{
    int index = ((int)program - (int)PCB_SET) / PCB_SIZE;
    PCB_SET_STATUS[index] = false;
    this->allPrograms.erase(&(program->tagInAllList));
    printf("pid %d has been recycled\n",index);
}

```

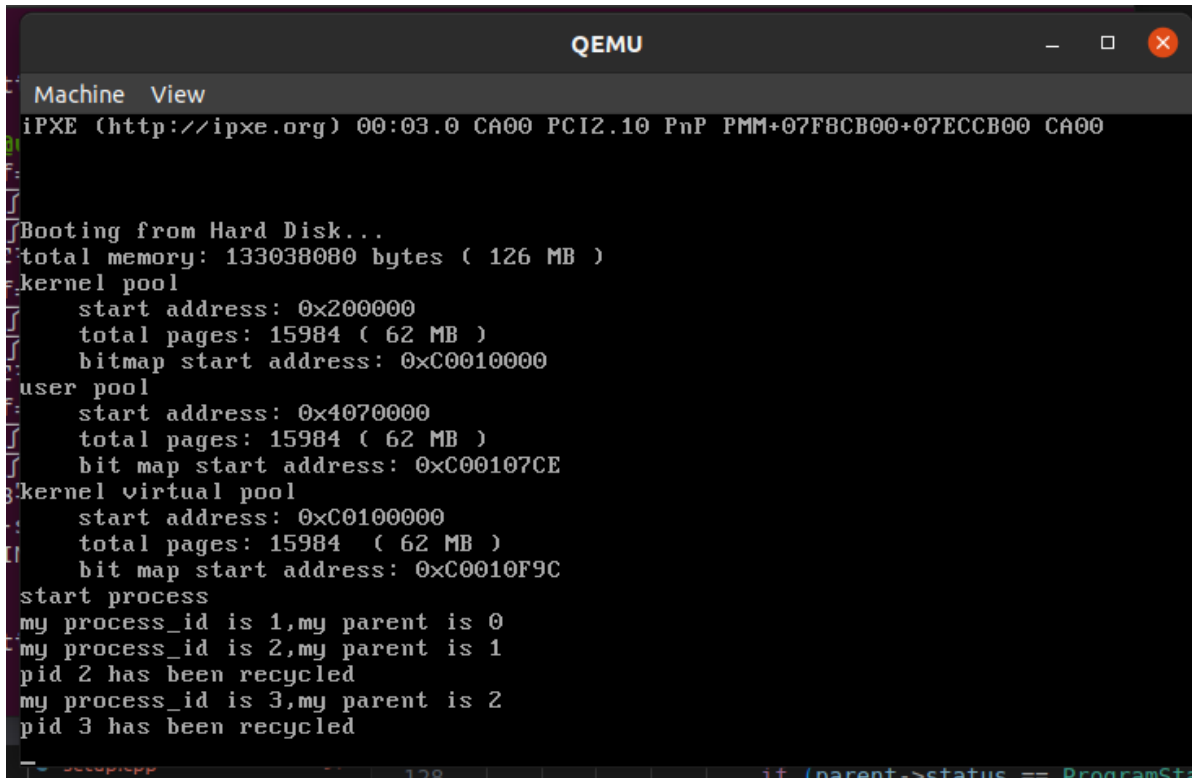
展示示例如下，我们在主进程（pid为1）中使用fork创建子进程（pid为2），在子进程里面再创建一个子进程（pid为3），主进程调用exit（0）退出，进程2调用exit(0)退出。那么因为主进程的父线程是主线程，主线程不会退出，所以主进程不会被回收，只会退出成为僵尸进程，而进程2的父进程是主进程，则进程2执行exit(0)退出成为僵尸进程，然后调度后被回收，打印信息，进程3的父进程是进程2（已经被回收），则进程3执行默认exit(0)成为僵尸进程，也会被回收，打印信息。这个示例包含了父进程是僵尸进程，父进程已经被回收两种情况。

```

// 第一种回收僵尸进程的示例
void first_process() {
    int pid = fork();
    if(pid) {
        printf("my process_id is %d,my parent is %d\n",programManager.running -> pid, programManager.running -> parentPid);
        exit(0);
    } else {
        int pid2 = fork();
        if(pid2) {
            printf("my process_id is %d,my parent is %d\n",programManager.running -> pid, programManager.running -> parentPid);
            exit(0);
        } else {
            printf("my process_id is %d,my parent is %d\n",programManager.running -> pid, programManager.running -> parentPid);
        }
    }
}

```

执行结果如下：



```
Machine View
iPXE (http://ipxe.org) 00:03.0 CA00 PCI2.10 PnP PMM+07F8CB00+07ECCB00 CA00

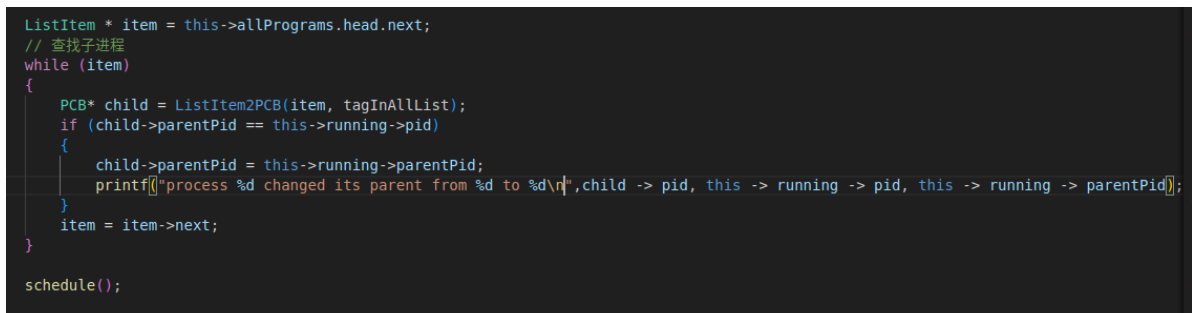
Booting from Hard Disk...
total memory: 133038080 bytes ( 126 MB )
kernel pool
  start address: 0x200000
  total pages: 15984 ( 62 MB )
  bitmap start address: 0xC0010000
user pool
  start address: 0x4070000
  total pages: 15984 ( 62 MB )
  bit map start address: 0xC00107CE
kernel virtual pool
  start address: 0xC0100000
  total pages: 15984 ( 62 MB )
  bit map start address: 0xC0010F9C
start process
my process_id is 1,my parent is 0
my process_id is 2,my parent is 1
pid 2 has been recycled
my process_id is 3,my parent is 2
pid 3 has been recycled
it (parent->status == ProgramSta
```

可以看到打印信息符合我们的预期，可以用于孤儿进程的回收。

第二种方法：

第二种方法思路是当一个进程退出时，修改其所有子进程的父进程为该进程的父进程，当fork操作完成后，父子进程除了共享代码段没有特别的联系，因此可以进行该操作，最终，所有的进程的父进程会变成主线程，我们在主线程中实现wait操作即可删除僵尸进程。

在exit()函数的最后进行该操作即可：



```
Listitem * item = this->allPrograms.head.next;
// 查找子进程
while (item)
{
    PCB* child = ListItem2PCB(item, tagInAllList);
    if (child->parentPid == this->running->pid)
    {
        child->parentPid = this->running->parentPid;
        printf("process %d changed its parent from %d to %d\n", child -> pid, this -> running -> pid, this -> running -> parentPid);
    }
    item = item->next;
}

schedule();
```

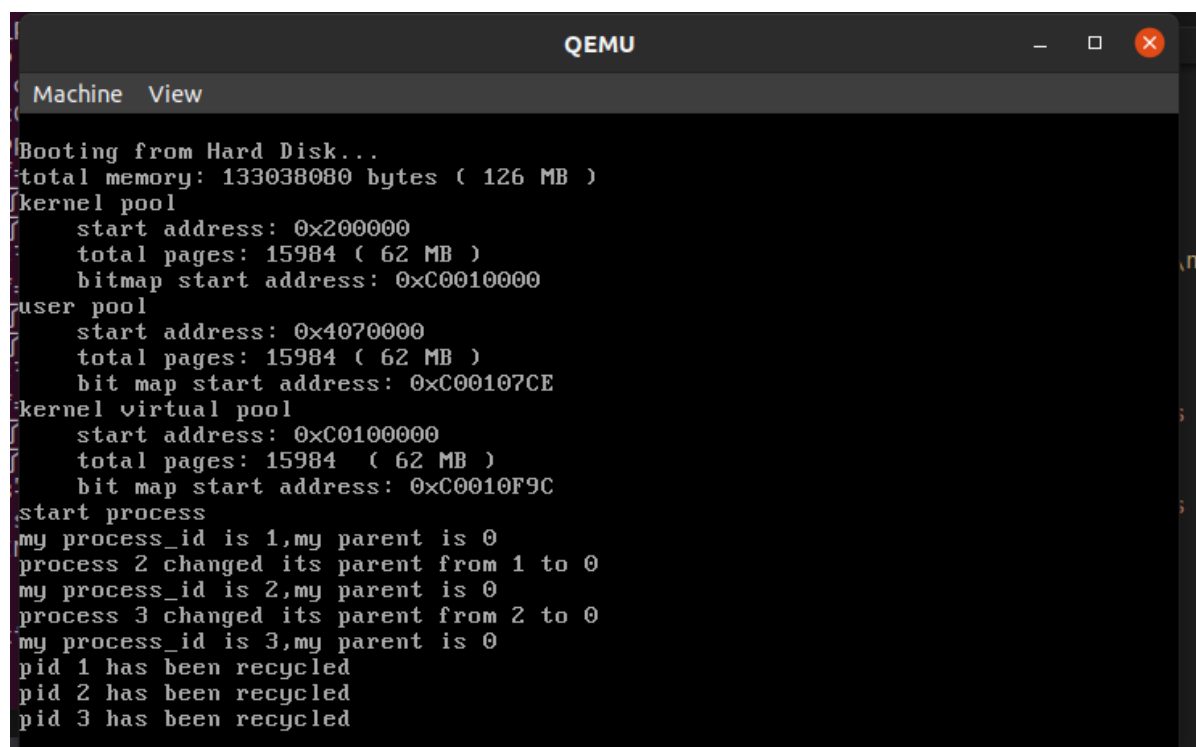
在实现用例与上面类似，不同的是，我们在主线程中设置一个死循环，该循环会不断尝试执行wait操作。当所有进程执行完成后，它们的父进程会变成主线程（pid = 0），所以在主线程的wait操作可以将全部的僵尸进程删除掉。这里我们为了防止后来创建的进程占据之前进程的PCB导致pid重复，在主线程中设置了一个延迟：

```

7 // 第二种回收僵尸孤儿进程的示例
8 void first_process() {
9     int pid = fork();
10    if(pid) {
11        printf("my process_id is %d,my parent is %d\n",programManager.running -> pid, programManager.running -> parentPid);
12        exit(0);
13    } else {
14        int pid2 = fork();
15        if(pid2) {
16            printf("my process_id is %d,my parent is %d\n",programManager.running -> pid, programManager.running -> parentPid);
17            exit(0);
18        } else {
19            printf("my process_id is %d,my parent is %d\n",programManager.running -> pid, programManager.running -> parentPid);
20        }
21    }
22 }
23
24 void second_thread(void *arg)
25 {
26     printf("thread exit\n");
27     //exit(0);
28 }
29
30 void first_thread(void *arg)
31 {
32     printf("start process\n");
33     programManager.executeProcess((const char *)first_process, 1);
34     // programManager.executeThread(second_thread, nullptr, "second", 1);
35     uint32 tmp = 0xffffffff;
36     while(tmp--);
37     while(true) {
38         wait(nullptr);
39     }
40     asm_halt();
41 }

```

执行效果如下，可以看到，所有的僵尸进程最后父亲都变成了主线程，然后在主线程中进行了回收。结果正确。



The screenshot shows the QEMU Machine View window. The output text is as follows:

```

Machine View
Booting from Hard Disk...
total memory: 133038080 bytes ( 126 MB )
kernel pool
  start address: 0x200000
  total pages: 15984 ( 62 MB )
  bitmap start address: 0xC0010000
user pool
  start address: 0x4070000
  total pages: 15984 ( 62 MB )
  bit map start address: 0xC00107CE
kernel virtual pool
  start address: 0xC0100000
  total pages: 15984 ( 62 MB )
  bit map start address: 0xC0010F9C
start process
my process_id is 1,my parent is 0
process 2 changed its parent from 1 to 0
my process_id is 2,my parent is 0
process 3 changed its parent from 2 to 0
my process_id is 3,my parent is 0
pid 1 has been recycled
pid 2 has been recycled
pid 3 has been recycled

```